

Scan your app, then shield it with a WAF

Session 8 – DevSecOps, WAF & secure apps

Exercise 8 · Session 8 lab · Estimated time: 90–120 min

Write your answers in the answer document `SKY2100_Exercise8_Answers.docx`. Your Nextcloud is encrypted (S6) and the host is hardened (S7). Now you treat it as an application to test and protect. You will run a dynamic (DAST) scan, check for outdated components, then put a web application firewall in front and watch it block a test attack.

Caution: Scan and attack only your own VM. Sending attack payloads at systems you do not own is illegal. The test payloads below are harmless probes against your own service.

1 Dynamic scan (DAST)

▷ Run an OWASP ZAP baseline scan against your HTTPS Nextcloud (Docker one-liner, or the ZAP GUI).

Use the VM's address from `ip a`, not `localhost`: inside the container, `localhost` is the container itself.

```
docker run -rm -t ghcr.io/zaproxy/zaproxy:stable \
zap-baseline.py -t https://<vm-ip>
```

▷ Skim the report. Name two findings and their risk level (e.g. missing security header, outdated component). You do not need to fix them all today.

2 Check your components

▷ Open Nextcloud's built-in Security & setup warnings (Administration settings → Overview) and note any warning or outdated component.

▷ Why is an outdated dependency an application-security risk, even if your code is fine?

3 Put a WAF in front (ModSecurity + OWASP CRS)

▷ Add ModSecurity with the OWASP Core Rule Set to your reverse proxy in front of Nextcloud.

A quick route is the official ModSecurity-CRS container, or the Nginx/Apache ModSecurity module. Start in detection mode (`SecRuleEngine DetectionOnly`), confirm the site still works, then switch to prevention (`SecRuleEngine On`).

▷ Why start in detection mode before turning on prevention? (one line)

4 Attack your own app — and watch it block

▷ Send a harmless test payload that the OWASP CRS should catch, e.g.:

```
curl -k "https://<your-host>/?q=<script>alert(1)</script>"
curl -k "https://<your-host>/?id=1' OR '1'='1"
```

▷ What HTTP status do you get with the WAF in prevention mode, and what was logged?

AHA. Think about what your earlier controls did and did not cover. TLS in Session 6 encrypted the connection, and the hardening in Session 7 locked down the host, yet the payload you just sent was a perfectly valid HTTPS request to a port you meant to keep open. Encryption and a firewall cannot see that the content is an attack, because to them it is normal traffic to an allowed service. That is why the attack moves up to the application layer, and why the WAF works there: it reads the request itself and blocks known bad patterns before they reach the app. Note also that the WAF matched a known pattern, so it is a guard, not a cure. A novel logic flaw it has no rule for would pass, which is why secure code and scanning still matter.

5 Cloud transfer: Microsoft Learn

▷ Read “Tutorial: Create an Application Gateway with a Web Application Firewall” and note how the WAF uses the OWASP rule set.

learn.microsoft.com/azure/web-application-firewall/ag/application-gateway-web-application-firewall-portal

▷ Azure WAF runs in detection or prevention mode. Which one actually protects the app, and why would you ever run the other?

6 Azure reflection

▷ In a DevSecOps pipeline, **SAST** runs in CI and **DAST** runs in CD. In one or two lines each, say what each test looks at and why both are needed.

▷ Your WAF blocked an attack at runtime. Name one application-security control from earlier in the SSDLC that should also address the same attack (defence in depth).

7 Knowledge check

1. Why does “the attack move up the stack” to the application after S6 and S7?

Hint: Transit (S6) and the host (S7) are locked, so the application is what remains.

2. Name three OWASP Top 10 categories.

Hint: Broken access control, injection, cross-site scripting — any three.

3. What is the common root cause of injection and XSS?

Hint: Untrusted input treated as code or commands.

4. Why is a vulnerable **dependency** your problem, even with perfect code?

Hint: Most of a modern app is third-party and open-source code.

5. List the five stages of the CSA Secure SDLC.

Hint: Design, coding, build/integrate/test, deliver/deploy, operate.

6. What is **threat modelling**, and when in the lifecycle is it done?

Hint: Finding and ranking threats — done early, during design.

7. Explain “shift left” and why it saves money.

Hint: Move security toward the start; a design flaw is far cheaper than a production one.

8. Define **SAST** and **DAST**, and say when each runs in CI/CD.

Hint: One reads source code before deployment; one probes the running app.

9. What is **SCA**, and what is an **SBOM**?

Hint: It audits your external dependencies for known vulnerabilities; the SBOM is their ingredient list.

10. What does a **WAF** do, and which OWASP resource backs its rules?

Hint: It inspects HTTP at layer 7; its rules come from the OWASP Core Rule Set.

11. Detection vs prevention mode: what is the difference?

Hint: One logs matching requests but lets them through; one blocks them.

12. Why is a WAF “a guard, not a cure”? Give one limitation.

Hint: It blocks known patterns — a novel logic flaw walks straight past.

Progress check: Discuss your answers in the group, then talk the teacher through them verbally at the next session only to track progress; nothing is handed in, signed or graded.

Solutions

Work through the lab first, then check here. Model answers are indicative — equivalent reasoning is fine.

Note: Model answers are indicative – accept equivalent reasoning. Multiple-choice answers are in **bold**.

Knowledge check

1. Each control raised the bar: TLS protects transit (S6) and hardening locks the host (S7), so the remaining target is the application logic. A valid HTTPS request to an open service can still carry an attack in its payload, which the firewall lets through.
2. Any three of: broken access control; injection (SQL/command); cross-site scripting (XSS); security misconfiguration; vulnerable/outdated components; identification & authentication failures; cryptographic failures; insecure design.
3. **Untrusted input treated as code/commands** – a SQL string, a shell command, or browser script. Fix by validating input and parameterising/encoding output.
4. Modern apps are mostly third-party and open-source code; a flaw in a dependency is a flaw in your app (e.g. Log4Shell). You ship the dependency, so you own its risk – hence SCA and an SBOM.
5. **Secure Design & Architecture; Secure Coding; Continuous Build, Integration & Testing; Continuous Delivery & Deployment; Runtime Defense & Monitoring** (CSA p. 231).
6. A structured process to identify, assess and prioritise threats to a system’s objectives. Done early, during design, when changing the answer is cheap (CSA p. 231–232).
7. “Shift left” moves security toward the start of the lifecycle. A flaw caught in design/code is far cheaper to fix than one found in production (CSA p. 233).
8. **SAST** reads source code at rest for flaws – runs pre-deployment in **CI**. **DAST** probes the running app as a black box – runs post-deployment in **CD**. Both are needed: code vs runtime (CSA p. 234–235, 242).
9. **SCA (Software Composition Analysis)** audits external/open-source dependencies for known vulnerabilities and licensing risk. An **SBOM (Software Bill of Materials)** lists all components shipped, giving transparency (CSA p. 234).
10. A **WAF** inspects HTTP requests at layer 7 and blocks known attack patterns (SQLi, XSS) before they reach the app. Its rules are based on the **OWASP Core Rule Set (CRS)**.
11. **Detection** logs matching requests but lets them through (for tuning); **prevention** actively blocks them. Only prevention protects the app.
12. It blocks known patterns and can be bypassed by novel logic flaws; it needs tuning (too strict → false positives, too loose → misses). It complements secure code, it does not replace it.

Lab checkpoints

- **DAST (ZAP):** the baseline scan typically flags missing security headers (CSP, HSTS), cookie flags, and sometimes outdated components. The point is to read a dynamic report – DAST is the post-deployment, black-box test (CSA p. 234).
- **Component check:** Nextcloud’s security/setup warnings stand in for SCA – outdated PHP, missing modules, or an out-of-date instance. The lesson: dependencies and configuration are part of the attack surface.
- **WAF in front:** ModSecurity + OWASP CRS should sit between the client and Nextcloud. Starting in DetectionOnly avoids blocking legitimate traffic while tuning; then SecRuleEngine

On enforces. This is the SSDLC runtime-defence stage.

- **Blocked attack:** the XSS/SQLi probes should return **403** in prevention mode, with a matching CRS rule ID in the audit log (e.g. a 941xxx XSS or 942xxx SQLi rule). Detection mode would log but still return 200 – a good contrast to show.

Cloud transfer & reflection

- **SAST vs DAST:** SAST reads source code for flaws/secrets in CI (pre-deployment); DAST probes the running app from the outside in CD (post-deployment). Both are needed because each has the other's blind spot – code internals vs real runtime behaviour (CSA p. 242).
- **Defence in depth:** the WAF blocks the injection/XSS at runtime, but the same attack should also be addressed earlier – secure coding (input validation, parameterised queries, output encoding) and SAST in CI. Fix it in the code and block it at the edge.
- **Detection vs prevention:** prevention blocks and protects; detection only logs. You run detection first to tune out false positives before enforcing, so real users are not blocked.

References

CSA Security Guidance v5, **Domain 10: Application Security** (p. 230–245, verified): SSDLC five stages (p. 231), threat modelling (p. 231–232), SAST/SCA/static scanning (p. 233–234), DAST/fuzzing/ IAST/pen test/bug bounty (p. 234–235), DevSecOps CI→SAST / CD→DAST and the six pillars (p. 242–244). Comer, The Cloud Computing Book, Ch. 15 – DevOps: the DevOps approach (p. 210), CI (p. 211), CD (p. 212), sandbox/canary/blue-green (p. 212–213), difficult aspects (p. 213–214), verified. MS Learn: Application Gateway with a Web Application Firewall (verified, June 2026) – OWASP CRS, detection vs prevention mode.